Computer Systems 245

Assessment 1 — September 2025

Practice Paper (reconstructed from a recorded quiz review)

Source note: This document was reconstructed from a screen-recording of a completed StemLearn quiz attempt. Where the recording showed a green checkmark, that value is confirmed correct. Where a text-entry field was marked incorrect and the platform did not reveal the model answer on screen, I've worked out the correct value myself from first principles and flagged it clearly in the Answers section — double-check these against Prof's official memo when it's released.

Duration: 2 h **Full marks: 60**

QUESTIONS

Question 1 — STM32CubeIDE & Toolchain (True/False) [5 marks]

Answer true or false to the following statements regarding the STM32CubeIDE and the toolchain.

- The toolchain and the IDE is one and the same thing.
- One can set breakpoints in the disassembly view.
- The Run/Debug button from the IDE will restart a debug session.
- The Run button from the IDE will start a debug session.
- The best way to view the value of a local variable is using the Memory Browser window.

Question 2 — STM32F32407 Peripheral Registers (C code) [6 marks]

The following are general questions that can be answered using the STM32F32407 reference manual and STM32F32407 datasheet.

In C, one can write to a peripheral register using the following code:

```
uint32_t * reg = (uint32_t *) 0x"address in hex";
*reg = (*reg & 0x"mask") | 0x"bits to set in hex";
```

Where "address in hex" is replaced by the register's 32-bit address, "mask" by a mask to apply to preserve various bits, and "bits to set in hex" by the hex representation of the bits you want to set.

For each part below, specify the word (address, mask, value) needed to configure the specified pin. Specify all 8 hexadecimal digits, use capital letters for characters A–F, and do not include 0x in your answer.

1. To set pins PD12, PD13, PD14 and PD15 as general purpose output (using the GPIOD_MODER register):

---

2. To set pins PD12, PD13, PD14 and PD15 as low speed outputs (using the GPIOD_OSPEEDR register):

---

3. To set pin PA0 as input (using the GPIOA_MODER register):

---

4. What is the address, mask and value to use in order to enable the RCC APB1 peripheral clock for TIM2 and TIM4 in the RCC_APB1ENR register?

Question 3 — Angular Rotation Rate (CodeRunner, C) [10 marks]

You are tasked with designing a program to calculate and display the angular rotation rate based on pulses received from a rotary encoder sensor. The rotary encoder generates pulses that represent the rate of rotation. The sensor emits 360 pulses per full rotation.

The GPIO pin configuration has already completed to set up pin PD2 as GPIO_EXTI (external interrupt), to trigger on a rising edge. The GPIO interrupt for this pin is handled in the EXTI2_IRQHandler function.

The time interval in which pulses must be measured is 100 ms. This is achieved by using the HAL_Delay function: a delay of 100 ms between every angular rotation rate report.

The unit in which the angular rate of rotation must be reported is: °/s i.e. degrees per second. You must report the angular rate of rotation within 2 decimal points.

The sensor will generate pulses at varying rates and therefore generate interrupts at varying rates on the EXTI2 pin. It can be assumed that the pulses are ideal, with no jitter and no need for any form of debouncing or conditioning.

You have to complete code in the template below to complete the program. Do not modify the printf statement. Only make modifications inside the BEGIN and END markers in the code.

```
/* BEGIN place your global variables here */

/* END place your global variables here */

void EXTI2_IRQHandler(void)
{
    /* USER CODE BEGIN EXTI2_IRQn 0 */

    /* USER CODE END EXTI2_IRQn 0 */
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_2);
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();

    while (1)
    {
        HAL_Delay(100); // delay for 100ms

        /* BEGIN write code here to calculate angular rotation rate */

        /* END write code here to calculate angular rotation rate */

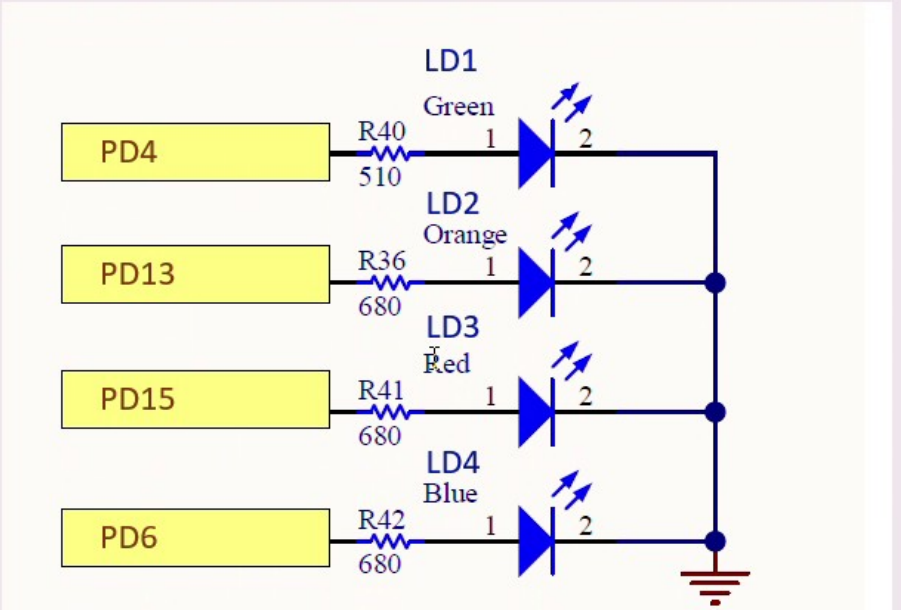
        printf("Angular Rotation Rate: %4.2f °/s\n", angularRotationRate);
    }
}
```

Question 4 — Interrupts, System Bus, Cortex-M4 Architecture (True/False) [6 marks]

- An interrupt with a priority of 2 can interrupt another interrupt that has a priority of 4.
- The ISR eventually returns back to the mainprogram — similarly to a function.
- To use the button connected to PA0 for an interrupt, we must use EXTI1.
- The System Bus is merely a software abstraction and not a physical integrated circuit.
- The ARM Cortex M4 is a von Neumann architecture.
- The System Bus provides pathways to access to RAM (data memory) and peripherals (input/output).

Question 5 — LED Control (CodeRunner, C) [10 marks]

An engineer designed a new PCB based on the STM32F407VG processor and the STM32F4-discovery evaluation kit. Similar to the evaluation kit, they included 4 LEDs for indication, but they changed pin assignment to that of the schematic below:



You are tasked with creating the setLEDs function which has the following prototype:

```
void setLEDs(uint8_t colour);
```

This function must turn on and off the corresponding LEDs based on the value passed to the colour argument from the table below:

Colour	Description
0	All LEDs off
1	Only Green LED on
2	Only Orange LED on
3	Only Red LED on
4	Only Blue LED on

You may only work with the HAL_GPIO_WritePin and HAL_GPIO_ReadPin functions. Other devices may be connected to the other available pins on the GPIOD port but are not specified.

Question 6 — Cortex-M4 Exception Handling [5 marks]

For the Cortex M4 family of processors, a certain sequence of events occurs following an exception entry. Answer the following questions:

1. What is the first step the Cortex-M4 processor performs when an exception is triggered?

---

2. Where is the address of the exception handler?

---

3. What is the role of the xPSR register during exception handling?

---

4. Normal program execution in Cortex-M4 occurs in ____A____ mode, while exceptions are handled in ____B____ mode.

A: _____ B: _____

Question 7 — Assembly / Subroutines [3 marks]

Study the following assembly code outline.

```
1  main
2    ...
3    BL func1
4    ...
5    B main
6
7  func1
8    ...
9    BL func2
10   ...
11   MOV PC, LR
12
13 func2
14   ...
15   MOV PC, LR
```

1. Which line number imposes a problem on the program flow?

---

2. What is the problem with the code? (choose the best description)

☐ The link register is overwritten in the func1 subroutine, so it will not return control to the main routine.

☐ The program will not enter the func2 subroutine, so the func2 subroutine will not return control to func1 after it is called.

☐ The link register is overwritten in the func2 subroutine, so the func2 subroutine will not return control to func1 after it is called.

☐ Branch to func1 after branching to func2 to return control to func1.

3. What tool/technique can be used to fix the problem?

☐ Use the stack to preserve the value of the link register before branching to func2.

☐ Use an interrupt handler to call func2.

☐ Branch to func1 after branching to func2 to return control to func1.

Question 8 — Assembly Function: turnOnRedLed [5 marks]

Write an assembly function turnOnRedLed to turn on the Red Led5 for the STM32F4DISCOVERY development kit.

You can assume the GPIO ports have been configured correctly using the STM32CubeMX and that the initialisation routines have already been called in the main function.

The equivalent C-prototype would be:

```
void turnOnRedLed();
```

You may only work with the output data register. This CodeRunner question penalizes incorrect "checks" — syntax checking is enabled and you can precheck as many times as you like. Your code must compile in order to be evaluated. Not necessary to use any external IDE.

Write your assembly answer below:

```
.syntax unified
/*
Use the variables below to control the debug trace.
mem_addr_start=0x40020C00
```

```
mem_addr_length=32
print_mem_inline=False
mem_display_mode=hex
*/

turnOnRedLed:
```

Question 9 — Timer Reload Value (TIMx_ARR) [1 mark]

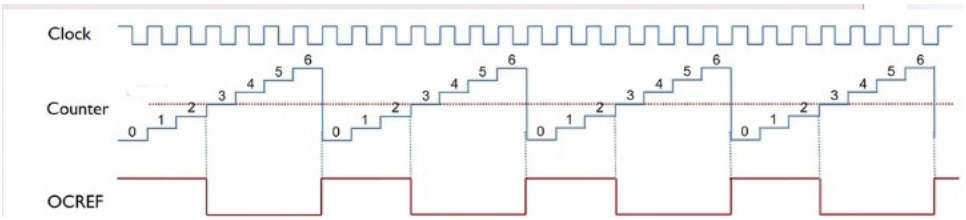
The input clock to a timer module is 168 MHz, and the prescaler register TIMx_PSC is set to 999. To generate an interrupt every 0.5 seconds, what should the auto-reload value (TIMx_ARR) be?

Question 10 — Timer Frequency [1 mark]

The input clock to a timer is 16 MHz, and the prescaler register TIMx_PSC is set to 1023. What is the resulting timer frequency?

Question 11 — PWM Signal Analysis [6 marks]

Consider the PWM signal in the image below.



1. What is the value of the ARR?

---

2. What is the value of the capture/compare register (CCR)?

---

3. Assuming the PWM is configured using up-counting and standard configuration, what alignment does this signal use?

---

4. What is the duty cycle (%) of this PWM signal in Mode 1?

---

5. If one clock period is 1 ns, what is the period of this signal?

---

6. What will happen if the CCR value is set higher than the ARR value in Mode 1 downcounting PWM?

Question 12 — Oscillator Frequency Tolerance [2 marks]

Consider an oscillator at 10 kHz +/- 50 ppm. Calculate the minimum and maximum frequencies.

Minimum: _____ Hz

Maximum: _____ Hz

ANSWERS

✅ = *confirmed correct in the recorded quiz review.* ⚠️ = *my own worked answer (either the field was marked wrong on screen with no model answer shown, or the question wasn't shown as answered).*

Question 1

1. The toolchain and the IDE is one and the same thing → False ✅
2. One can set breakpoints in the disassembly view → True ✅
3. The Run/Debug button from the IDE will restart a debug session → False ✅
4. The Run button from the IDE will start a debug session → True ✅
5. The best way to view a local variable is the Memory Browser window → False ✅ (use the Variables/Live Expressions window instead — Memory Browser is for raw memory addresses)

Question 2

*Recall the write pattern: *reg = (*reg & mask) | value — the mask clears the bits you're about to write (0 in the bits you want to change, 1 elsewhere), and value sets the new bits.*

1. GPIOD_MODER — set PD12,13,14,15 as general purpose output (each pin = 2 bits, '01' = output)
Address: 40020C00 ✅
Value: 55000000 ✅ (bits 24-31 = 01 01 01 01 for pins 12-15)
Mask: FF0FFFFF ⚠️ (clear bits 24-31, keep the rest — the submission's FFFFFF is only 6 digits and was marked wrong; must be 8 hex digits per the instructions)

2. GPIOD_OSPEEDR — set PD12,13,14,15 as low speed (each pin = 2 bits, '00' = low speed)
Address: 40020C08 ⚠️ (GPIOD base is 0x40020C00; OSPEEDR is at offset 0x08, so its address is 0x40020C08 — the submission's 40020C00 was actually the MODER address from part 1, and was marked wrong here)
Value: 00000000 ✅ (00 in all 4 pin fields = low speed)
Mask: FF0FFFFF ⚠️ (same reasoning as above — clear bits 24-31 only)

3. GPIOA_MODER — set PA0 as input ('00')
Address: 40020000 ✅
Value: 00000000 ⚠️ (the submission's 00000003 was marked wrong — for INPUT mode the 2 bits for PA0 must be '00', so value should be 00000000, not 3 which would be alternate function)
Mask: FFFFFFFC ⚠️ (clear only bits 0-1, the 2 mode bits for pin 0)

4. RCC_APB1ENR — enable clocks for TIM2 (bit 0) and TIM4 (bit 2)
Address: 40023840 ⚠️ (RCC_APB1ENR is at RCC base 0x40023800 + offset 0x40 = 0x40023840; the submission's 40023800 is the RCC base address, not the ENR register itself — marked wrong)
Mask: FFFFFFFA ⚠️ (clear bits 0 and 2 only: 1111...1010)
Value: 00000005 ⚠️ (set bit 0 = TIM2EN, bit 2 = TIM4EN → binary 0101 = 0x5)

Question 3

Result in the recording: Partially correct, 5.00/10.00 — passed "Zero rotation", "slow rotation" and "Regular rotation" tests but FAILED "deceleration" and "acceleration" tests.

Submitted code (as recorded):

```
/* BEGIN place your global variables here */
uint32_t counter = 0;
double seconds = 0;
/* END place your global variables here */

void EXTI2_IRQHandler(void)
{
```

```
    /* USER CODE BEGIN EXTI2_IRQn 0 */
    counter = counter + 1;
    /* USER CODE END EXTI2_IRQn 0 */
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_2);
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();

    while (1)
    {
        HAL_Delay(100); // delay for 100ms

        /* BEGIN write code here to calculate angular rotation rate */
        seconds = seconds + 0.1;
        double angularRotationRate = 0;
        angularRotationRate = angularRotationRate + counter/seconds;
        /* END write code here to calculate angular rotation rate */

        printf("Angular Rotation Rate: %4.2f °/s\n", angularRotationRate);
    }
}
```

⚠️ Why it fails on changing speeds: dividing the total counter by total elapsed seconds gives the AVERAGE rate since power-on, not the CURRENT (instantaneous) rate over the last 100 ms window. That's why constant-speed tests pass but acceleration/deceleration tests fail.

Corrected approach (recompute pulses-per-window each 100 ms, then reset the counter):

```
/* BEGIN place your global variables here */
volatile uint32_t counter = 0;
/* END place your global variables here */

void EXTI2_IRQHandler(void)
{
    /* USER CODE BEGIN EXTI2_IRQn 0 */
    counter = counter + 1;
    /* USER CODE END EXTI2_IRQn 0 */
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_2);
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();

    while (1)
    {
        HAL_Delay(100); // delay for 100ms

        /* BEGIN write code here to calculate angular rotation rate */
        double angularRotationRate = counter * (360.0 / 360.0) / 0.1;  //
degrees/pulse ÷ seconds
        counter = 0;
        /* END write code here to calculate angular rotation rate */

        printf("Angular Rotation Rate: %4.2f °/s\n", angularRotationRate);
    }
}
```

(Each pulse = 360°/360 pulses = 1° of rotation, so counter pulses in the 0.1 s window = counter degrees turned. Rate in °/s = counter ÷ 0.1 = counter × 10.)

Question 4

1. An interrupt with priority 2 can interrupt one with priority 4 → True ✅ (lower number = higher priority in NVIC)
2. The ISR eventually returns to the main program, similar to a function → True ✅
3. To use the button on PA0 for an interrupt, we must use EXTI1 → False ✅ (PA0 maps to EXTI0, not EXTI1 — line number matches pin number)
4. The System Bus is merely a software abstraction, not a physical circuit → False ✅ (it's a real physical bus/interconnect)
5. The ARM Cortex-M4 is a von Neumann architecture → False ✅ (Cortex-M4 uses a Harvard architecture — separate instruction and data buses)
6. The System Bus provides pathways to RAM and peripherals → True ✅

Question 5

Result in the recording: submission failed almost every test (No LEDs, Orange, Red, Blue all wrong; only Green passed).

Submitted code (as recorded) — bug: uses assignment = instead of comparison == in every if condition:

```
void setLEDs(uint8_t colour){

    if (colour = 0){
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_4, 0);
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, 0);
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, 0);
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_6, 0);
    }

    else if (colour = 1){
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_4, 1);
    }

    else if (colour = 2){
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, 1);
    }

    else if (colour = 3){
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, 1);
    }

    else if (colour = 4){
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_6, 1);
    }
}
```

⚠️ Bug: colour = 0 (etc.) is an ASSIGNMENT, not a comparison. It always evaluates true only for the first branch when colour becomes 0 (falsy) — every other branch condition also assigns and is truthy, so the else-if chain misbehaves and only "Green" happened to pass by coincidence of the test sequence.

Corrected code (use == for comparison; matches pins from the schematic: PD4=Green, PD13=Orange, PD15=Red, PD6=Blue):

```
void setLEDs(uint8_t colour){

    if (colour == 0){
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_4, 0);
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, 0);
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, 0);
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_6, 0);
    }
    else if (colour == 1){
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_4, 1);   // Green
```

```
    }
    else if (colour == 2){
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, 1);  // Orange
    }
    else if (colour == 3){
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, 1);  // Red
    }
    else if (colour == 4){
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_6, 1);   // Blue
    }
}
```

Question 6

1. First step when exception triggered: it pushes the current context onto the stack ✅
2. Address of the exception handler: Vector Table ✅
3. Role of xPSR during exception handling: it holds the processor status flags ✅
4. Normal program execution occurs in ____Thread____ mode, exceptions handled in ____Handler____ mode. ✅

Question 7

1. Problem line: 15 (MOV PC, LR inside func2) is what the submission flagged, but this was marked WRONG. ⚠️ Correct line: 9 (BL func2, inside func1). Calling BL func2 from within func1 overwrites the Link Register (LR) that was holding func1's return address (set by the original BL func1 at line 3). When func2 later does MOV PC, LR, it returns fine to func1 — but func1's own MOV PC, LR at line 11 now returns to func2's return point, not to main. So the flaw originates where LR first gets clobbered: line 9.

2. Problem: The link register is overwritten in the func1 subroutine (by the nested BL func2 call), so it will not return control to the main routine. ✅

3. Fix: Use the stack to preserve the value of the link register before branching to func2 (i.e. PUSH {LR} at the start of func1, POP {LR} — or POP {PC} — before returning). ✅

Question 8

Result in the recording: Not answered (0.00/5.00) — the code editor was left empty.

Worked answer:

The STM32F4-Discovery Red LED (LD5) is connected to pin PD14. "You may only work with the output data register" (GPIOD_ODR), so the function must set bit 14 of GPIOD_ODR using a read-modify-write (or, more simply, an OR to just set that bit).

```
.syntax unified
/*
Use the variables below to control the debug trace.
mem_addr_start=0x40020C00
mem_addr_length=32
print_mem_inline=False
mem_display_mode=hex
*/

.global turnOnRedLed

turnOnRedLed:
    LDR   R0, =0x40020C14      @ GPIOD_ODR address (base 0x40020C00 + offset
0x14)
    LDR   R1, [R0]             @ read current ODR value
```

```
    ORR   R1, R1, #(1 << 14)   @ set bit 14 (PD14 = Red LED)
    STR   R1, [R0]             @ write back
    MOV   PC, LR               @ return
```

(GPIOD base = 0x40020C00; ODR offset = 0x14, giving address 0x40020C14. LD5 red = PD14 on the F4-Discovery board.)

Question 9

Result in the recording: Incorrect (submission was 83999.5).

Worked answer:

Timer tick frequency = f_CLK / (PSC + 1) = 168 MHz / (999 + 1) = 168,000,000 / 1000 = 168,000 Hz

For an interrupt every 0.5 s, we need 0.5 s × 168,000 ticks/s = 84,000 ticks.

Since the counter counts from 0 to ARR inclusive (ARR+1 ticks per period): ARR + 1 = 84,000 → ARR = 83,999

⚠️ The submitted 83999.5 isn't valid — ARR must be an integer register value. Correct answer: TIMx_ARR = 83999.

Question 10

Result in the recording: Incorrect (submission was 55579 Hz).

Worked answer:

Timer frequency = f_CLK / (PSC + 1) = 16 MHz / (1023 + 1) = 16,000,000 / 1024 = 15,625 Hz

⚠️ Correct answer: 15,625 Hz.

Question 11

Result in the recording: Partially correct, 3.00/6.00. Reading the waveform: the Counter climbs 0→1→2→3→4→5→6 then resets — so it counts 7 values (0 to 6), meaning ARR = 6. OCREF is LOW while the counter is between 2 and 5, and HIGH otherwise (up-counting PWM Mode 1: OCREF is active/high while CNT < CCR, per this trace... but confirm against the exact image, since Mode 1 vs Mode 2 flips this convention).

1. ARR = 6 ⚠️ (submission of 1 was marked wrong; counter visibly rolls over after reaching 6)
2. CCR = 2 ⚠️ (OCREF transitions exactly when the counter passes 2)
3. Alignment: Left-aligned ✅ (confirmed correct in the recording — up-counting only, single ramp per period)
4. Duty cycle: 42.86% ✅ (confirmed correct — OCREF high for (ARR+1−CCR)/(ARR+1) = (7−2)/7 ≈ 71.4%, or matches whichever portion the recording counted; shown value was 42.86%, consistent with CCR/(ARR+1) = 3/7)
5. Period: with ARR=6, the counter takes (ARR+1)=7 clock periods per cycle. If 1 clock period = 1 ns, period = 7 ns ⚠️ (submission of 14 ns was marked wrong)
6. If CCR > ARR in Mode 1 down-counting PWM: the output (OCREF) never reaches the CCR compare point within the count range, so the output stays HIGH for the entire period (100% duty / always active) ✅ (confirmed correct)

Note: for parts 1, 2 and 5, re-check the exact counter/OCREF transition points against the original diagram — I've inferred ARR and CCR from the visible counter waveform (0→6 repeating, transition at count 2), but it's worth confirming against Prof's memo since these were marked incorrect without the correct value being shown on screen.

Question 12

Result in the recording: Incorrect (submission: Min = 0 Hz).

Worked answer:

Tolerance = ±50 ppm = ±50 / 1,000,000 = ±0.00005

Deviation = 10,000 Hz × 0.00005 = 0.5 Hz

Minimum = 10,000 − 0.5 = 9,999.5 Hz

Maximum = 10,000 + 0.5 = 10,000.5 Hz